Week 2

✅ Computer Organization and OS

- Computer-System Operation
- Common Functions of Interrupts

✅ Categories of OS

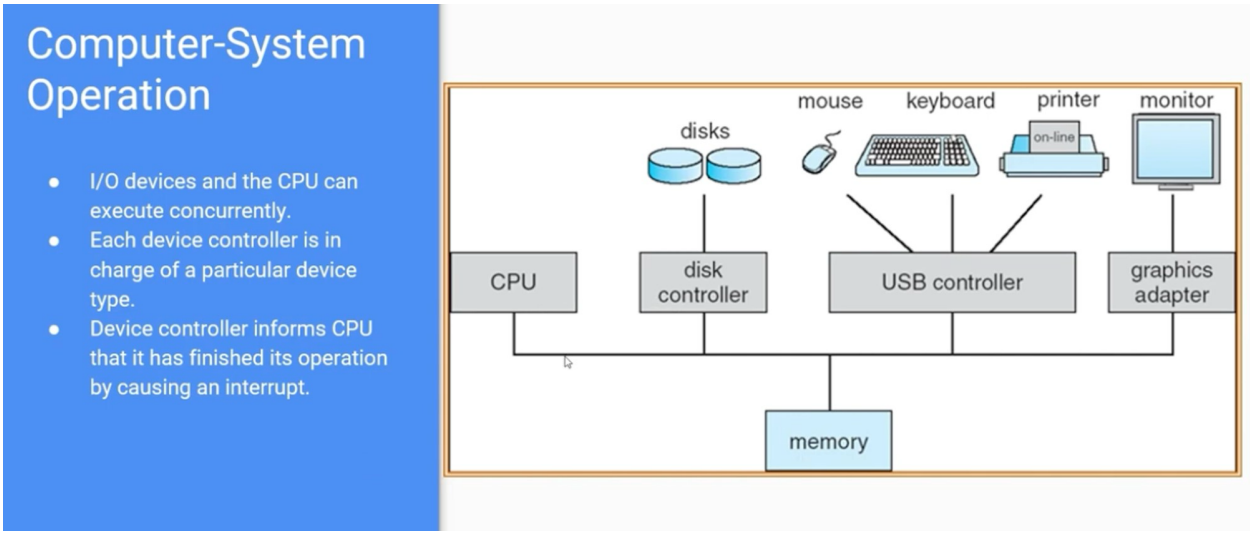
        - Criteria
    - How the data is entered into the system
        - Batch
            - Single program Batch System
            - Multi program Batch System
        - Interactive
        - Hybrid
    - Number of users & Tasks, OS can support
        - Single-user single-tasking
        - Single-user multi-tasking
        - Multi-user multi-tasking
- Special Purpose Systems
    - Real Time Systems
        - Hard Real Time
        - Soft Real Time
    - Embedded Systems
- Type of architecture of the computer
    - Single Processor
    - Parallel Systems(Multiprocessors)
    - Distributed Systems
        - Network OS (NOS)
        - Distributed OS (DOS)
        - Comparison of NOS and DOS

> **Content**

✅ Computer Organization(hardware) and OS

how an os and computer hardware work together?

📍 Computer-System Operation



This picture show overview of computer organization, as you can see CPU and controller of each devices.
Ex: Peripheral devices communicate through their usb controller.

- The idea is when data is request for some program → CPU executes instructions → CPU send the command “hey, I need data from disk” → disk controller get this command and read data from disk → data may moved to the buffer of controller first → data send to memory → CPU get data from memory
- However, In modern computer system, I/O devices and CPU can execute concurrently.

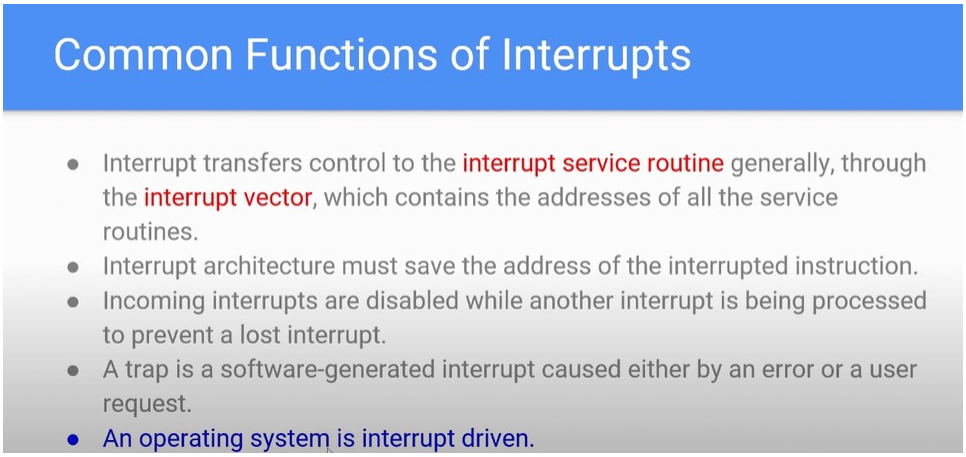
👉 Assuming that OS allows program A to run, when program run it use CPU, and then assuming that program A said that “at this point, I need data from disk” → data must be read from disk first before it continues.

- The easy way for OS and system→ wait until it finish getting data, then continue — not efficient, disk work very slow compared to cpu, nowadays speed rate of cpu is gigahertz/ second.
- (The new computer system) when program A need to use some I/O, program A get put to wait somewhere → OS choose program B to run, at this time CPU work concurrently (CPU execute program B, disk controller read data for program A)

> What if disk controller finish reading data for program A, it must have the way to inform the system like “hey, i finish my jobs” or else program A has to wait forever→ way for I/O to inform CPU is by**“causing an interrupt”.**

- When interrupt occur, OS will see what does interrupt mean and decide what to do, for example → “data for A is ready now, OS may allow A to wait in a queue of ready to run, OS may choose A after B finish, B need some I/O, or OS might interrupt B…..”

📍 Common Functions of Interrupts



An OS is interrupt driven

- As we know when interrupt occur → OS work
- if no interrupt → OS sleep, if interrupt occur → OS wake up

Vocab

- interrupt service routine (interrupt handler)
  - A program to handle interrupt, each hardware in a system has it own interrupt service routine (keyboard, mouse, disk, etc.)
  - This program is in a special area reserved by OS
  - Each hardware has it own unique number → Ex. no.6 is keyboard, so when interrupt come from keyboard, the no.6 is send to the system. → OS wake up
- interrupt vector (lookup table)
  - OS use the number to take a look at lookup table.
  - OS use interrupt number to search inside interrupt vector to find out the address for interrupt service routine for keyboard → run the program at that address to handle keyboard interrupt

How OS handle interrupt when it wakes up?

- When interrupt occur, OS just find the proper interrupt service routine, load and run it.

- However, when interrupt occur → there is a program running → when OS want to run interrupt service routine, OS have to make sure that it doesn't do anything wrong with the interrupted program → OS need to save everything related to interrupted program (data, place it was interrupted)
    - why? when interrupt service routine finish → it can continue where it was interrupted without any problem.
- What if an OS is handling interrupt and there is an new interrupt coming
    - differ/mask → OS will set up that OS will not process any interrupt at this time → interrupt get out into queue.
- Interrupt can come from software (trap)
    - occur when your program has some error.
        - user program require hardware, user program need to make request to the OS.

✅ Categories of OS

📍 **Criteria** we can divide or group OS using many criteria.

- How the data is entered into the system.
- Number of users and Tasks, OS can support.
- Special Purpose Systems.
- Type of architecture of the computer.

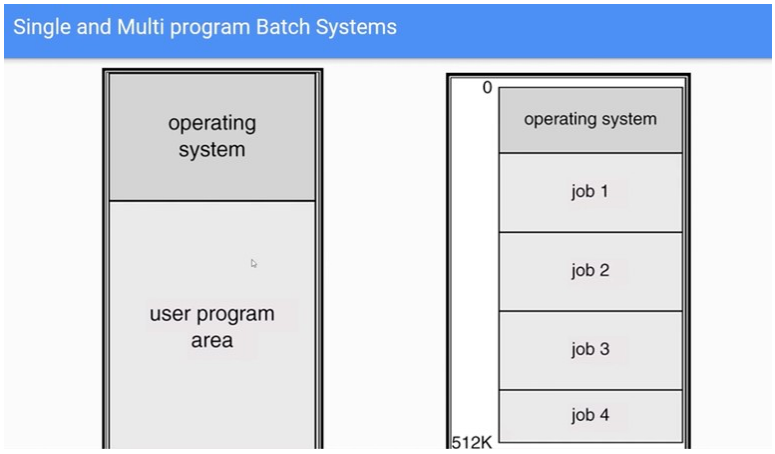
📐 How the data is entered into the system

- **Batch** - we need to provide all data for the program, the program not allow user to enter more data → all data must be provided maybe in file format→ OS run the program, read the data from file and process the result for you.

  💡 The jobs are processed serially, without user interaction

  - First rudimentary operating system.
    - **Automatic job sequencing** (basic task for this OS)- automatically transfer control from one job to another. — job or program will be loaded into computer, maybe there a queue for this jobs → OS choose the first one in a queue, and run, finish, choose next one in a queue.
  - Resident monitor
    - initial control in monitor (control system)
    - control transfers to job (load program, give control to that program) (finish)
    - when job completes control transfers back to monitor (OS back to monitor mode, take a look at a queue)

  - There are 2 Type in Batch System



1. Single programmed batch system - the explanation above, it's the task of single programmed batch
   - It can only handled 1 user program at a time. Each time, there can be only 1 user program in memory running.

- Problem → not efficient → 1 program at a time, and also when this program require I/O, then the system need to stop running the program to complete the I/O request of the program and then continue.

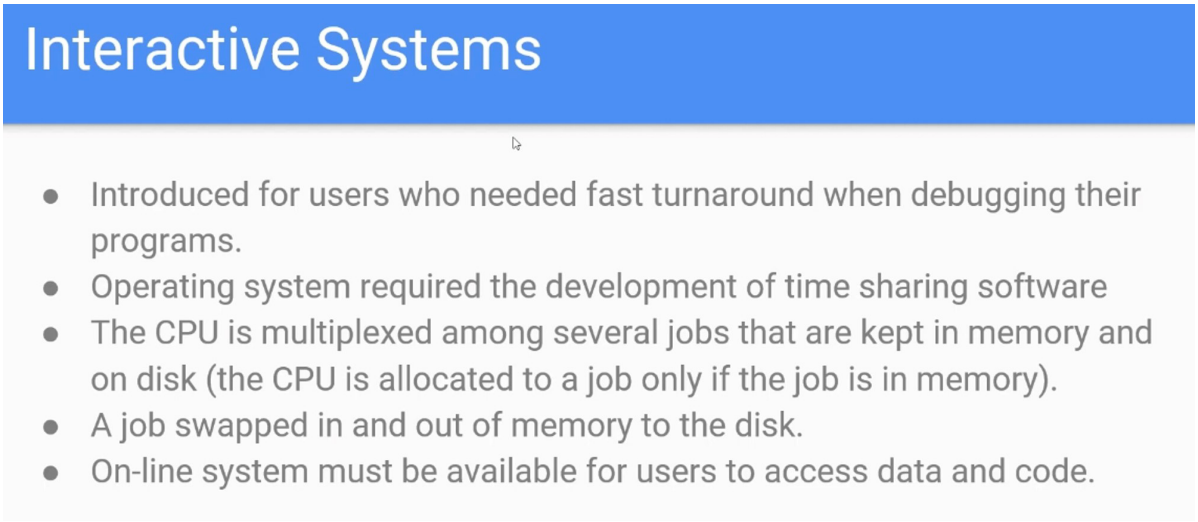
2. Multi programmed batch system (improved)

    - There are more than 1 user program that can be run at the same time. Ex: Job 1 is running and it require I/O to read the file, while jobs 1 waiting for info from file, OS can run jobs 2 → ……
    - Each jobs when it run, it'll run without user interaction.

> The Multi programmed batch is more efficient that a single program batch (not waste CPU time, but more complex than single programmed batch).

As we can see one thing that make it more complex is the way the protection of memory, for single programmed-batch, OS need to protect memory from user program area to OS area, this means that user program shouldn't access the OS area. For multi-programmed batch, ofcourse we have to protect OS area, but we have to protect each other too. Like job2 should not be interfere with memory of job1.

- **Interactive** - we can interact to computer, computer may prompt you 1 data and you enter the data, the computer may ask you for more data. (not all program are suitable for batch system, some program need to get the responds faster, maybe the way the major program that push the new system to be develop maybe the programming task when you want to debug the program, you would want to find out fast whether your program have some bug) — if we using batch program → we would have to provide program and data and put the program into the system, and the program wait in the queue until it have chance to run and then print out the result. You would have to wait very long → 1 day, half of day ◦ Therefore, we need OS that support interactive system.



  - ◦ allow user to interact with the system, user don't have to prepare everything, data and wait for the result.
  - ◦ Important feature of Interactive Systems OS is **time sharing** Time sharing → the sharing of cpu time.
  - ◦ To achieved this time sharing, the system must set the time slot that each program is allow to run for each round. And the time slot must set to be very fast, maybe milisecond

👉 When program A is allow to run by OS, it run by only in just this time slot, when time slot expired, if program A has finished, it exist. If program A can't finish in this time slot, then the system will interrupt program A and put program A to wait for the next round and then allow program B to run in time slot,……

  - ◦ when time slot expired, the timer interrupt was send(when interrupt occur, OS works), so OS then interrupt the running program and put that program into a queue and allow the next program to

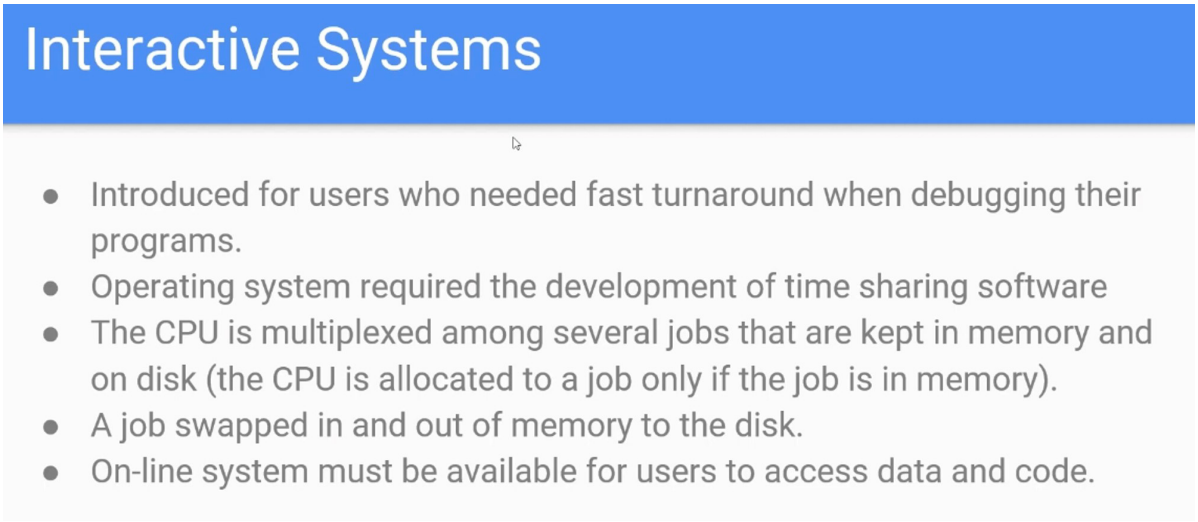
run. (the idea is similiar to what we learn - save the state of program, so when it come back, it can continue)

- the detail on how os switch from one program to another with be discuss with more detail later.
- The CPU is multiplexed among several jobs that are kept in memory and on disk (the cpu is allocated to a job only if the job is in memory)
  - The system must allow more than 1 program to be loaded at the memory at the same time. → This require the same feature as multi-batch system to protect memory area of each program and also protect memory area of the OS. When the time slot expired, OS will choose only the program that are in the memory to run next.

> What if the memory is full, and there're some program in the disk, then this program can't be run until the memory is clear (some program finish running)?

  - No, actually, this system is more complicate than that. (OS has something called job swapped)
  - A job swapped in and out of memory to the disk. — OS may swap some program from memory to disk and bring some program from disk to memory.
- On-line system must be available for users to access data and code. — allow user to use computer in interactive manner (type something on the keyboard, result appear on screen, system ask us to enter more command or data).

- **Hybrid** - OS that support both batch and interactive ◦ OS that we use today.
  - ◦ The major idea is the batch program that doesn't require any user interaction, we'll put it int he background, and then it may have lower priorities than the interactive program. — when OS has to choose between batch program and interactive program, OS may choose interactive program first.



  - ◦ Ex, when you use microsoft word and print 1000 page document to printer. Os may design to put your printing program in the background, and allow you to still do interactive task.

📐 Number of users & Tasks, OS can support

- **Single-user single-tasking**
  - ◦ only one user can use a computer at a time, and user can only run 1 program at a time.
  - ◦ when you turn you r machine on you are the only user, and you not allow to log on to the same machine from the network, and then you can run only 1 program at a time.
  - ◦ For example, if you run word processing program, and you type something, and then you want to use calculator, then you have to save the document to disk first, close you r word program and run your calculator, calculated, closed calculator, and run word again, ….
  - ◦ Ex: DOS

- **Single-user multi-tasking**
  - only one user at a time, but user can run more than 1 programs at a same time.
  - You can run word and calculator at the same time.
  - Ex: MS Windows, MAC OS X
  - something you may confused: user account → this allow only one user to use computer at a time, account that you create is unrelevant.
- **Multi-user multi-tasking**
  - more than 1 user to log on at a time, and system support more than 1 programs running at the same time
  - Ex: Unix (Server version of OS) (Windows Server)

📐 Special Purpose System

- **Real Time System** - The system must respond within the scope of time requirement. If the system can't respond within that time period, the system may consider failed.
  - Hard Real Time - System that if it can't respond within time requirement, it may cause serious damage or risk of life.
    - Avoid the device that work slow just like hard drive, it may use SSD, RAM, ROM to store the data.
    - This conflict with Time Sharing, so there's no time sharing system, not supported by general-purpose operating systems. → We can't use the OS that we use today, we have to specific OS that develop for this real-time system.

-
    - Ex: Lynx RTOS
  - Soft Real Time - The time is still important, but if the system can't respond within the time requirement → It may not cause serious damage like hard real time
    - When you watch the video, but it's not smoothly. (Not dangerous) (Not cause your life)
    - To support this, we can use the OS that we use today, OS may have features that give more priorities to soft real time system than normal application.

Embedded Systems

- Computers placed inside other products to add features and capabilities
- OS with small kernel and flexible functions capabilities will have potential for embedded system.
- For ex: Microwave, you can choose program to cook or warm your food
- Idea → the device that we gonna develop software to control, we gonna have limited amount of hardware (limited screen size, RAM, storage,…).
- For this system, we think about OS that can be work with limited amount of hardware.
- In the past, mobile phone may fall directly into this categories, because it has limited amount of hardware. (มอถอแตกอนทาไรไดไมเยอะ maybe store contact, send text message,…)
- Today → era of smart device → mobile phone, tablet has more ram and storage. However, mobile phone still have small screen, so the OS need to handled this small screen. It depends on touch features → OS has to pay more attention to this touch features than laptop or desktop.

📐 Type of architecture of the computer.

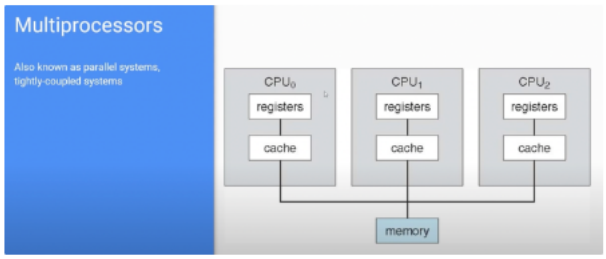
- **Single Processor** - in the past, it mean that the computer only have 1 cpu, and the cpu have only 1 core.

-
    - (already obsolete) nowadays, we have 1 cpu but multicore.

-

📐 Type of architecture of the computer.

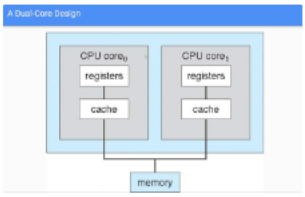
- **Single Processor** - in the past, it mean that the computer only have 1 cpu, and the cpu have only 1 core.
  - (already obsolete) nowadays, we have 1 cpu but multicore.
  - (Some concept that we going to explain, we still stick with this Single Processor)
- **Parallel Systems (Multiprocessors)**



  - In 1 computer, we have more than 1 cpu and they share the same hardware (memory, hard drive). Because of this sharing, sometimes, we called this tightly-coupled systems.
  - sometimes, we called it tightly-coupled systems - this mean the os that control this type of system must use the cpu efficiently → distributed the program to run in each cpu, at the same time, protect the hardware (the program running in many cpu)
  - → OS must have more complex features than the OS from Single Processor

---

A Dual-Core Design (Single processor Multicore Design)

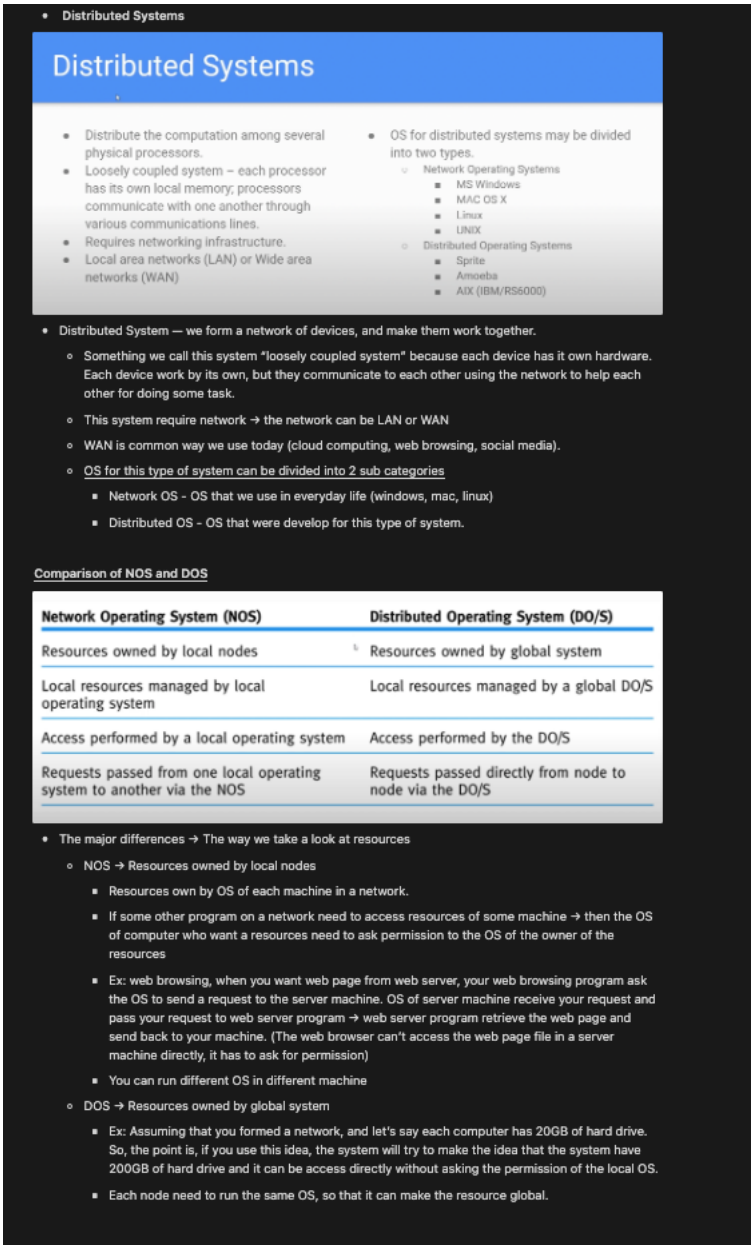


In this system, 1 processor can have more than 1 core (in this case, 2) → OS need to distributed the program to run in each core. And at the same time, it has to protect the hardware from the program that running in different core as well

- You may thing this is very similar to Multi processor system. That's right → But something to be noted.

> The communication between each core is faster than communication between each cpu.

> If core inside processor failed → CPU failed → System can't continue. For multiprocessor, if 1 cpu failed, other cpu can still be used to make the system still working.

- 
Distributed Systems
Distributed Systems
Distribute the computation among several physical processors.
Loosely coupled system – each processor has its own local memory; processors communicate with one another through various communications lines.
Requires networking infrastructure.
Local area networks (LAN) or Wide area networks (WAN)
OS for distributed systems may be divided into two types.
Network Operating Systems
MS Windows
MAC OS X
Linux
UNIX
Distributed Operating Systems
Sprite
Amoeba
AIX (IBM/RS6000)
Distributed System — we form a network of devices, and make them work together.
Something we call this system "loosely coupled system" because each device has it own hardware. Each device work by its own, but they communicate to each other using the network to help each other for doing some task.
This system require network → the network can be LAN or WAN
WAN is common way we use today (cloud computing, web browsing, social media).
OS for this type of system can be divided into 2 sub categories
Network OS - OS that we use in everyday life (windows, mac, linux)
Distributed OS - OS that were develop for this type of system.
Comparison of NOS and DOS
Network Operating System (NOS)
Distributed Operating System (DO/S)
Resources owned by local nodes
Resources owned by global system
Local resources managed by local operating system
Local resources managed by a global DO/S
Access performed by a local operating system
Access performed by the DO/S
Requests passed from one local operating system to another via the NOS
Requests passed directly from node to node via the DO/S
The major differences → The way we take a look at resources
NOS → Resources owned by local nodes
Resources own by OS of each machine in a network.
If some other program on a network need to access resources of some machine → then the OS of computer who want a resources need to ask permission to the OS of the owner of the resources
Ex: web browsing, when you want web page from web server, your web browsing program ask the OS to send a request to the server machine. OS of server machine receive your request and pass your request to web server program → web server program retrieve the web page and send back to your machine. (The web browser can't access the web page file in a server machine directly, it has to ask for permission)
You can run different OS in different machine
DOS → Resources owned by global system
Ex: Assuming that you formed a network, and let's say each computer has 20GB of hard drive. So, the point is, if you use this idea, the system will try to make the idea that the system have 200GB of hard drive and it can be access directly without asking the permission of the local OS.
Each node need to run the same OS, so that it can make the resource global.